

may be different). What has happened here is that *cloud.map* has created 100 jobs with the arguments formed from the 100 pairs of the two sequences, *r* and *h*. You can pass this list of *jids* directly to *cloud.result* to get the results as a list:

```
>>> result=code.result(jids)
>>> result
[0.35045338986267927, 0.0043144690799585004,
0.094018119621969785, 0.93579722612839329,
0.0045154147876736109, 0.018836324478609345,
0.0027243595262778321, 1.5049675511377265, 0.37383416636274164,
0.24435487403102638, 0.28248315493781553, 1.2879340600324913,
0.68406526971923841, 0.14338739850272786, ...]
```

In more practical situations, the function computation may not be so trivial, and hence you wouldn't have a definite idea when all the jobs will be over. In such a case, you can wait till all the jobs have finished to retrieve the results, using the function *cloud.join* (<http://docs.picloud.com/module-doc.html#cloud.join>)

## Cloud files

PiCloud's S3 data store can be used for persistent data storage on the cloud. The interface to this store is provided by the *cloud.files* module (<http://docs.picloud.com/module-doc.html#module-cloud.files>). The *cloud.files.list* method can be used to display all the files in your account:

```
>>> cloud.files.list()
[ ]
```

Since you haven't yet stored any files on PiCloud, an empty list is returned. To store a file on the cloud, the *cloud.files.put* method is used (this basically transfers the file from your local disk to the cloud):

```
>>> cloud.files.put('stats_1.csv')
>>> cloud.files.list()
['stats_1.csv']
```

This file can be retrieved using the *cloud.files.get* method:

```
>>> cloud.files.get('stats_1.csv')
```

There are methods for checking the existence of a file, deleting a file and opening a file on the cloud. There are also methods for *syncing files*, which would transfer a file from/to the cloud only if the file has changed—especially useful when you have large files to work with. A simple example to show how you would go about storing files on the PiCloud is shown below. Create a file *filedemo.py* as shown:

```
import cloud

def savedata():
    f=open('data.txt','w')
    f.write('This is a line of text')
    f.close()

cloud.files.put('data.txt')
```

Now, in the interpreter, use the following code:

```
>>> from filedemo import *
>>> cloud.call(savedata)
459
>>> cloud.files.list()
['data.txt', 'stats_1.csv']
>>> cloud.files.get('data.txt')
```

As you can see, the file '*data.txt*' has been created in the cloud, and can later be retrieved using *cloud.files.get*. This is useful when you run code that generates data that you need to examine.

## Publishing functions via REST

An interesting capability that PiCloud offers is to *publish* functions on the cloud via a REST interface.

TechnoMail - Enterprise Email Server  
Anti SPAM, Anti Virus, Email Content Filtering

Firewall, Internet Access Control  
Content Filtering, Site Blocking

Bandwidth Management System

Managed Email Hosting Solutions

# TechnoInfotech®

1, Vikas Permisses, 11 Bank Street, Fort, Mumbai, India - 400 001. Mobile: 09167399917. [info@technoinfotech.com](mailto:info@technoinfotech.com)